AEUP · PROGRAMMIEREN

C# Lernzettel

Grundlagen: Datentypen, Variablen, Ein- und Ausgabe und Typkonvertierung

#WIT10c #AEuP #Wagner

FACH	LEHRKRAFT	DATUM
AEuP	Svenja Wagner	15.10.2026

Inhalt

1 Konsolenprojekt und Grundgerüst 2

2 Datentypen (einfache) 3

3 Variablen 4

4 Ausgabe 4

5 Eingabe und Typkonvertierung 5

- Implizit und explizit 5

6 Beispiel: Trainingsplanung (Fitnessstudio) 6

7 Typische Prüfungsfallen 7

- Deklarationen 7
- Initialisierungen 7
- Ausgabe-Rätsel 8

8 Kurz-Checkliste 8

Alle Einträge sind anklickbar. Über „↑ Inhalt“ gelangst du von jedem Kapitel zurück hierher. Die Kurz-Checkliste am Ende lässt sich direkt im PDF abhaken.

KAPITEL 1

↑ Inhalt

1 Konsolenprojekt und Grundgerüst

- Visual Studio: Neues Projekt → Konsolen-App (C#) → Name vergeben → Erstellen
- Starten: `F5` (mit Debugger) oder `Strg + F5` (ohne Debugger)

```C#
internal class Program
{
    static void Main(string[] args)
    {
        // Hier steht dein Code
        Console.ReadKey(); // Fenster bleibt offen, bis eine Taste gedrückt wird
    }
}
```

Kommentare

- Einzeilig: `// Kommentar`
- Mehrzeilig: `/* Kommentar */`
- Kommentare werden vom Compiler ignoriert.

KAPITEL 2

↑ Inhalt

2 Datentypen (einfache)

Typ	Speicher	Wertebereich / Inhalt	Beispiel (Literal)
`int`	4 Byte	ganze Zahlen ca. ±2,1 Mrd.	`int alter = 28;`
`double`	8 Byte	Kommazahlen, 15-16 Stellen	`double d = 1.25;`
`string`	variabel	Zeichenkette	`string s = "Hallo";`
`char`	2 Byte	ein Unicode-Zeichen	`char c = 'x';`
`bool`	1 Bit	`true` / `false`	`bool ok = true;`
`long`	8 Byte	sehr große ganze Zahlen	`long l = 5000000000L;`
`float`	4 Byte	Kommazahl, 7-8 Stellen	`float f = 3.14f;`
`decimal`	16 Byte	Kommazahl, 28 Stellen (Geld)	`decimal m = 9.99m;`
`short`	2 Byte	-32.768 … 32.767	`short z = 15;`
`byte`	1 Byte	0 … 255	`byte b = 200;`

Anführungszeichen und Literale

- `"Text"` = doppelte Anführungszeichen → **string**
- `'x'` = einfache Anführungszeichen → **char**
- Kommazahlen mit Punkt schreiben: `1.25`
- Endungen: `f` = float, `m` = decimal, `L` = long

Welchen Typ nehme ich?

- Rechnen mit ganzen Zahlen → `int`
- Rechnen mit Kommazahlen → `double` (Geld: `decimal`)
- Zahlen, mit denen man **nicht rechnet** (Telefonnummer, PLZ, Hausnummer) → `string`, denn führende Nullen und `+49` gehen sonst verloren
- Ja/Nein-Information → `bool`

KAPITEL 3

↑ Inhalt

3 Variablen

Bezeichner (Name):

- erlaubt: Buchstaben, Ziffern, `_`
- nicht mit einer Ziffer beginnen, keine Leerzeichen, kein Schlüsselwort (z. B. `byte`, `int`)
- Groß-/Kleinschreibung zählt, üblich ist **camelCase**: `zahlDerVersuche`

Vorgang	Syntax	Beispiel
Deklarieren	`Typ name;`	`int alter;`
Initialisieren	`name = Wert;`	`alter = 28;`
Beides gleichzeitig	`Typ name = Wert;`	`int alter = 28;`
Mehrere gleichen Typs	`Typ a, b;`	`int x, y;`

> ✎ **Reihenfolge**
>
> Immer erst der **Typ**, dann der **Bezeichner**: `int alter;`, nicht `alter int;`
> Der Wert steht rechts vom `=`: `char c = 'A';`, nicht `char 'A' = c;`

KAPITEL 4

↑ Inhalt

4 Ausgabe

```C#
string vorname = "Anna";
int alter = 28;

Console.WriteLine("Hallo!");                                  // Text
Console.WriteLine(alter);                                     // Variable
Console.WriteLine("Name: " + vorname + ", Alter: " + alter);  // Verkettung mit +
Console.WriteLine($"Name: {vorname}, Alter: {alter}");        // String-Interpolation
```

Variante	Merkmal
Verkettung `+`	Text und Variablen mit `+` verbinden
`$"…{variable}…"`	übersichtlicher, `$` vor dem Text nicht vergessen

- `bool` wird als `True` / `False` ausgegeben.

KAPITEL 5

↑ Inhalt

5 Eingabe und Typkonvertierung

`Console.ReadLine()` liefert immer einen string. Für Zahlen muss konvertiert werden.

Von → Nach	Befehl
string → int	`int.Parse(s)` oder `Convert.ToInt32(s)`
string → double	`double.Parse(s)` oder `Convert.ToDouble(s)`
Zahl → string	`zahl.ToString()`

```C#
Console.WriteLine("Wie alt bist du?");
string eingabe = Console.ReadLine();
int alter = Convert.ToInt32(eingabe);   // kurz: int alter =
int.Parse(Console.ReadLine());
Console.WriteLine($"Nächstes Jahr bist du {alter + 1}.");
```

FormatException

Ist der Text keine gültige Zahl (z. B. `"abc"`), stürzt das Programm mit einer `FormatException` ab.

Sicherer: TryParse *(nicht besprochen, aber sinnvoll)*

```C#
bool possible = double.TryParse("123", out double number);
// possible = true  → number enthält 123
// possible = false → keine Exception, number = 0
```

Rückgabe `true` / `false` zeigt, ob die Umwandlung geklappt hat. Vorher mit `if` prüfen.

Implizit und explizit

- **Implizit** (automatisch, kein Datenverlust): kleiner → größer, z. B. `short → int`, `float → double`, `int → long`
- **Explizit** (Cast, Verlust möglich): größer → kleiner, Typ in Klammern davorsetzen

```C#
short x = 123;
int y = x;          // implizit, ok
int gross = 100;
byte klein = (byte)gross;   // explizit, ohne (byte) → Fehler
```

Rangfolge laut Unterricht: decimal > double > float > long > int > short > byte

KAPITEL 6

↑ Inhalt

6 Beispiel: Trainingsplanung (Fitnessstudio)

Aufgabe

Mitgliederdaten als Variablen mit passendem Datentyp anlegen, direkt initialisieren und in mehreren Sätzen ausgeben.

Information	Variable	Datentyp	Begründung
Vorname, Name	`vorname`, `nachname`	`string`	Text
Alter	`alter`	`int`	ganze Zahl
Gewicht	`gewicht`	`double`	Kommazahl
Vorerkrankung	`hatVorerkrankung`	`bool`	ja / nein
Trainingsziel	`trainingsziel`	`string`	Text
Telefonnummer	`telefonnummer`	`string`	keine Rechenzahl, führende 0
E-Mail, Adresse	`eMail`, `adresse`	`string`	Text
Bankverbindung	`iban`	`string`	Buchstaben und Ziffern
Eintrittsdatum	`eintrittsdatum`	`string`	hier als Text (`DateTime` später)

```C#
string vorname = "Anna";
string nachname = "Muster";
int alter = 28;
double gewicht = 62.5;
bool hatVorerkrankung = false;
string trainingsziel = "Muskelaufbau";
string telefonnummer = "0151 1234567";

Console.WriteLine($"Mitglied: {vorname} {nachname}, {alter} Jahre alt.");
Console.WriteLine($"Gewicht: {gewicht} kg, Ziel: {trainingsziel}.");
Console.WriteLine("Vorerkrankung: " + hatVorerkrankung);
Console.WriteLine($"Erreichbar unter {telefonnummer}.");
```

KAPITEL 7

↑ Inhalt

7 Typische Prüfungsfallen

Deklarationen

Zeile	Bewertung	Grund
`int zahl_der_Versuche;`	ok	Unterstrich erlaubt, besser `zahlDerVersuche`
`boolean istVerheiratet;`	falsch	in C# heißt es `bool`
`float postleitzahl;`	falsch	PLZ → `string`
`long kantenlaenge;`	unpassend	Kante kann Kommazahl sein → `double`
`short byte;`	falsch	`byte` ist ein Schlüsselwort
`int nummer, anzahl;`	ok	mehrere Variablen gleichen Typs
`long telefonnummer; hausnummer;`	falsch	`hausnummer` hat keinen Typ
`nummer byte;`	falsch	Typ muss vorn stehen, `byte` ist reserviert
`byte i, j;`	ok	

Initialisierungen

Zeile	Bewertung	Korrekt / Grund
`int zahlDerVersuche = 15;`	ok	
`double gehalt = 2645.34€;`	falsch	`double gehalt = 2645.34;` (kein `€`)
`int hausnummer = 24;`	ok	bei „24a" wäre `string` nötig
`char 'A' = buchstabe;`	falsch	`char buchstabe = 'A';`
`byte b = 324;`	falsch	`byte` nur 0 … 255
`short z = 15;`	ok	
`boolean istVerheiratet = false;`	falsch	`bool istVerheiratet = false;`
`double laenge = breite = hoehe;`	falsch	`breite` und `hoehe` müssen vorher deklariert und befüllt sein
`long postleitzahl = 02365;`	unpassend	führende 0 geht verloren → `string "02365"`
`float umfang = 64537.34756;`	falsch	Literal ist `double` → `64537.34756f` (oder `double`)

Ausgabe-Rätsel

```C#
Console.WriteLine(0);          // 0
int zero = 0;
Console.WriteLine(zero);       // 0
int one = 1;
int whatsThis = zero;
Console.WriteLine(whatsThis);  // 0
whatsThis = one;
Console.WriteLine(whatsThis);  // 1
```

Ausgabe: `0`, `0`, `0`, `1`. Die Zuweisung kopiert den Wert, und `whatsThis` wird danach überschrieben.

KAPITEL 8

↑ Inhalt

8 Kurz-Checkliste

- [ ] Typ vor Bezeichner, Wert rechts vom `=` → Kapitel 3
- [ ] `"…"` für string, `'…'` für char → Kapitel 2
- [ ] Dezimalpunkt statt Komma im Code → Kapitel 2
- [ ] Zahlen ohne Rechenzweck → `string` → Kapitel 2
- [ ] `ReadLine()` gibt immer string zurück → konvertieren → Kapitel 5
- [ ] Jede Anweisung endet mit `;` → Kapitel 1